

Migrácie a Modely – Hudobná škola

MIGRÁCIE

2024_01_01_000001_create_users_table.php

php

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('username')->unique();
    $table->string('email')->unique()->nullable();
    $table->timestamp('email_verified_at')->nullable();
    $table->string('password');
    $table->string('role')->default(UserRole::Student->value);
    $table->boolean('must_change_password')->default(true);
    $table->date('birth_date')->nullable();
    $table->rememberToken();
    $table->timestamps();
});

Schema::create('password_reset_tokens', function (Blueprint $table) {
    $table->string('email')->primary();
    $table->string('token');
    $table->timestamp('created_at')->nullable();
});

Schema::create('sessions', function (Blueprint $table) {
    $table->string('id')->primary();
    $table->foreignId('user_id')->nullable()->index();
    $table->string('ip_address', 45)->nullable();
    $table->text('user_agent')->nullable();
    $table->longText('payload');
    $table->integer('last_activity')->index();
});
```

2024_01_01_000002_create_school_years_table.php

php

```
// Skolský rok ako samostatná entita - na ňu sa odkazuje všetko ostatné
Schema::create('school_years', function (Blueprint $table) {
    $table->id();
    $table->string('name'); // napr. "2024/2025"
    $table->date('start_date');
    $table->date('end_date');
    $table->boolean('is_active')->default(false);
    $table->timestamps();
});
```

2024_01_01_000003_create_departments_table.php

```
php

// Odbor: hudba, tanec, maľba, ...
Schema::create('departments', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->text('description')->nullable();
    $table->timestamps();
});
```

2024_01_01_000004_create_specializations_table.php

```
php

// Špecializácia: gitara, klavír, spoločenské tance, ...
// Nepotrebuje school_year - špecializácia sa nemení každý rok
Schema::create('specializations', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->foreignId('department_id')->constrained()->cascadeOnDelete();
    $table->timestamps();
});
```

2024_01_01_000005_create_rooms_table.php

```
php
```

```
Schema::create('rooms', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->unsignedSmallInteger('capacity')->nullable();
    $table->boolean('reservable')->default(true);
    $table->text('description')->nullable();
    $table->boolean('is_active')->default(true);
    $table->timestamps();
});
```

2024_01_01_000006_create_teachers_table.php

```
php

Schema::create('teachers', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->unique()->constrained()->cascadeOnDelete();
    $table->foreignId('specialization_id')->nullable()->constrained()->nullOnDelete();
    $table->foreignId('department_id')->nullable()->constrained()->nullOnDelete();
    $table->foreignId('room_id')->nullable()->constrained()->nullOnDelete();
    $table->string('employee_number')->unique()->nullable();
    $table->timestamps();
});
```

2024_01_01_000007_create_students_table.php

```
php

Schema::create('students', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->unique()->constrained()->cascadeOnDelete();
    $table->foreignId('specialization_id')->nullable()->constrained()->nullOnDelete();
    $table->string('student_number')->unique()->nullable();
    $table->timestamps();
});
```

2024_01_01_000008_create_guardians_table.php

```
php
```

```
Schema::create('guardians', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->unique()->constrained()->cascadeOnDelete();
    $table->string('phone')->nullable();
    $table->timestamps();
});
```

2024_01_01_000009_create_guardian_student_table.php

```
php

// Zákonný zástupca môže mať viac detí, dieťa môže mať viac zákonných zástupcov
Schema::create('guardian_student', function (Blueprint $table) {
    $table->foreignId('guardian_id')->constrained()->cascadeOnDelete();
    $table->foreignId('student_id')->constrained()->cascadeOnDelete();
    $table->string('relationship')->nullable(); // matka, otec, starý rodič, ...
    $table->primary(['guardian_id', 'student_id']);
});
```

2024_01_01_000010_create_classes_table.php

```
php

// Trieda/skupina žiakov
Schema::create('classes', function (Blueprint $table) {
    $table->id();
    $table->string('name'); // napr. "I.A", "Gitara začiatoč
    $table->foreignId('school_year_id')->constrained()->cascadeOnDelete();
    $table->foreignId('teacher_id') // triedny učiteľ
        ->nullable()->constrained('teachers')->nullOnDelete();
    $table->foreignId('room_id') // kmeňová trieda
        ->nullable()->constrained()->nullOnDelete();
    $table->timestamps();
});
```

2024_01_01_000011_create_class_student_table.php

```
php
```

```
Schema::create('class_student', function (Blueprint $table) {
    $table->foreignId('class_id')->constrained()->cascadeOnDelete();
    $table->foreignId('student_id')->constrained()->cascadeOnDelete();
    $table->primary(['class_id', 'student_id']);
});
```

2024_01_01_000012_create_subjects_table.php

```
php

// Všeobecná definícia predmetu - existuje nezávisle od školského roka
Schema::create('subjects', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->text('description')->nullable();
    $table->foreignId('department_id')->nullable()->constrained()->nullOnDelete();
    $table->foreignId('specialization_id')->nullable()->constrained()->nullOnDelete();
    // napr. "Hra na gitaru" → Odbor: Hudba + Špecializácia: Gitara
    $table->timestamps();
});
```

2024_01_01_000013_create_subject_school_years_table.php

```
php

// Predmet v konkrétnom školskom roku pre konkrétnu triedu
// Toto je kľúčová entita - na ňu sa viažu hodiny, zápisy, hodnotenia
Schema::create('subject_school_years', function (Blueprint $table) {
    $table->id();
    $table->foreignId('subject_id')->constrained()->cascadeOnDelete();
    $table->foreignId('school_year_id')->constrained()->cascadeOnDelete();
    $table->foreignId('teacher_id')->constrained('teachers')->cascadeOnDelete();
    $table->foreignId('class_id')->nullable()->constrained()->nullOnDelete();
    $table->string('name_override')->nullable(); // voliteľný iný názov ako má Predm
    $table->timestamps();

    $table->index(['subject_id', 'school_year_id']);
});
```

2024_01_01_000014_create_time_slots_table.php

2024_01_01_000015_create_lessons_table.php

```
php

// Hodina = konkrétny výskyt vyučovania (jeden deň, jeden čas)
Schema::create('lessons', function (Blueprint $table) {
    $table->id();
    $table->foreignId('subject_school_year_id')->constrained()->cascadeOnDelete();
    $table->foreignId('room_id')->nullable()->constrained()->nullOnDelete();
    $table->foreignId('time_slot_id')->nullable()->constrained()->nullOnDelete();
    $table->date('date');
    $table->string('state')->default(LessonState::Planned->value);
    $table->text('note')->nullable();
    $table->timestamps();

    $table->index(['subject_school_year_id', 'date']);
    $table->index(['room_id', 'date']);
});
```

2024_01_01_000016_create_subject_enrollments_table.php

```
php
```

```
// Zápis žiaka na predmet v školskom roku (nahrádza ZápisNaPredmet + ŠtúdiumŽiaka)
Schema::create('subject_enrollments', function (Blueprint $table) {
    $table->id();
    $table->foreignId('student_id')->constrained()->cascadeOnDelete();
    $table->foreignId('subject_school_year_id')->constrained()->cascadeOnDelete();
    $table->string('state')->default(EnrollmentState::Active->value);
    $table->date('enrolled_at');
    $table->text('note')->nullable();
    $table->timestamps();

    $table->unique(['student_id', 'subject_school_year_id']); // žiak sa nemôže zapí
});
```

2024_01_01_000017_create_attendances_table.php

```
php

// Dochádzka žiaka na konkrétnu hodinu
Schema::create('attendances', function (Blueprint $table) {
    $table->id();
    $table->foreignId('student_id')->constrained()->cascadeOnDelete();
    $table->foreignId('lesson_id')->constrained()->cascadeOnDelete();
    $table->string('state')->default(AttendanceState::Present->value);
    $table->text('note')->nullable();
    $table->timestamps();

    $table->unique(['student_id', 'lesson_id']); // jeden záznam na hodinu na žiaka
});
```

2024_01_01_000018_create_grade_events_table.php

```
php
```

```
// Písomka / projekt / skúška - udalosť, z ktorej sa udeľujú známky
Schema::create('grade_events', function (Blueprint $table) {
    $table->id();
    $table->foreignId('subject_school_year_id')->constrained()->cascadeOnDelete();
    $table->foreignId('lesson_id')->nullable()->constrained()->nullOnDelete();
    $table->string('name');
    $table->text('description')->nullable();
    $table->string('type')->default(GradeEventType::Test->value);
    $table->date('date')->nullable();
    $table->timestamps();
});
```

2024_01_01_000019_create_grades_table.php

```
php

// Hodnotenie žiaka - vždy viazané na PredmetVŠkolskomRoku
// Voliteľne aj na Hodinu a GradeEvent
Schema::create('grades', function (Blueprint $table) {
    $table->id();
    $table->foreignId('student_id')->constrained()->cascadeOnDelete();
    $table->foreignId('teacher_id')->constrained('teachers')->cascadeOnDelete();
    $table->foreignId('subject_school_year_id')->constrained()->cascadeOnDelete();
    $table->foreignId('lesson_id')->nullable()->constrained()->nullOnDelete();
    $table->foreignId('grade_event_id')->nullable()->constrained()->nullOnDelete();
    $table->string('value'); // "1","2",... alebo "A","B",... alebo "výbo
    $table->string('type')->default(GradeType::Ongoing->value);
    $table->text('note')->nullable();
    $table->timestamp('graded_at')->useCurrent();
    $table->timestamps();

    $table->index(['student_id', 'subject_school_year_id']);
});
```

2024_01_01_000020_create_notes_table.php

```
php
```



```
// Poznámka učiteľa k žiakovi (pochvala, pokarhanie, ...)
Schema::create('notes', function (Blueprint $table) {
    $table->id();
    $table->foreignId('student_id')->constrained()->cascadeOnDelete();
    $table->foreignId('teacher_id')->constrained('teachers')->cascadeOnDelete();
    $table->foreignId('lesson_id')->nullable()->constrained()->nullOnDelete();
    $table->text('content');
    $table->string('type')->default(NoteType::Neutral->value);
    $table->boolean('visible_to_guardian')->default(true);
    $table->timestamps();
});
```

2024_01_01_000021_create_instruments_table.php

```
php

Schema::create('instruments', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('brand')->nullable();
    $table->string('model')->nullable();
    $table->string('serial_number')->nullable()->unique();
    $table->string('condition')->default(InstrumentCondition::Good->value);
    $table->foreignId('specialization_id')->nullable()->constrained()->nullOnDelete();
    $table->foreignId('room_id')->nullable()->constrained()->nullOnDelete(); // kde
    $table->boolean('is_available')->default(true);
    $table->text('notes')->nullable();
    $table->timestamps();
});
```

2024_01_01_000022_create_room_reservations_table.php

```
php
```

```
Schema::create('room_reservations', function (Blueprint $table) {
    $table->id();
    $table->foreignId('room_id')->constrained()->cascadeOnDelete();
    $table->foreignId('reserved_by')->constrained('users')->cascadeOnDelete();
    $table->string('purpose')->default(ReservationPurpose::Lesson->value);
    $table->dateTime('start_at');
    $table->dateTime('end_at');
    $table->text('note')->nullable();
    $table->string('status')->default(ReservationStatus::Pending->value);
    $table->timestamps();

    $table->index(['room_id', 'start_at', 'end_at']);
});
```

2024_01_01_000023_create_instrument_reservations_table.php

php

```
Schema::create('instrument_reservations', function (Blueprint $table) {
    $table->id();
    $table->foreignId('instrument_id')->constrained()->cascadeOnDelete();
    $table->foreignId('reserved_by')->constrained('users')->cascadeOnDelete();
    $table->string('purpose')->default(ReservationPurpose::Home->value);
    $table->dateTime('start_at');
    $table->dateTime('end_at');
    $table->text('note')->nullable();
    $table->string('status')->default(ReservationStatus::Pending->value);
    $table->timestamps();

    $table->index(['instrument_id', 'start_at', 'end_at']);
});
```

2024_01_01_000024_create_bands_table.php

php

```
// Školská kapela / súbor
Schema::create('bands', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->foreignId('teacher_id')->nullable()->constrained('teachers')->nullOnDelete();
    $table->unsignedSmallInteger('capacity')->nullable();
    $table->text('description')->nullable();
    $table->timestamps();
});
```

2024_01_01_000025_create_band_student_table.php

php

```
Schema::create('band_student', function (Blueprint $table) {
    $table->foreignId('band_id')->constrained()->cascadeOnDelete();
    $table->foreignId('student_id')->constrained()->cascadeOnDelete();
    $table->primary(['band_id', 'student_id']);
});
```

2024_01_01_000026_create_events_table.php

php

```
// Koncerty, výlety, súťaže, ...
Schema::create('events', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('type')->default(EventType::Concert->value);
    $table->dateTime('starts_at');
    $table->dateTime('ends_at')->nullable();
    $table->foreignId('room_id')->nullable()->constrained()->nullOnDelete();
    $table->string('location')->nullable(); // pre výlety / externé miesta
    $table->text('description')->nullable();
    $table->boolean('is_public')->default(false);
    $table->timestamps();

    $table->index(['type', 'starts_at']);
});
```

2024_01_01_000027_create_event_participants_table.php

php

```
Schema::create('event_participants', function (Blueprint $table) {
    $table->foreignId('event_id')->constrained()->cascadeOnDelete();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->string('role')->default(EventParticipantRole::Attendee->value);
    $table->text('note')->nullable();
    $table->primary(['event_id', 'user_id']);
});
```

2024_01_01_000028_create_event_band_table.php

php

```
// Kapela vystupuje na koncerte
Schema::create('event_band', function (Blueprint $table) {
    $table->foreignId('event_id')->constrained()->cascadeOnDelete();
    $table->foreignId('band_id')->constrained()->cascadeOnDelete();
    $table->primary(['event_id', 'band_id']);
});
```

ENUMY

```
app/Enums/
├─ UserRole.php           (admin, student, teacher, guardian)
├─ LessonState.php       (planned, taught, cancelled)
├─ AttendanceState.php   (present, absent, excused, late)
├─ EnrollmentState.php   (active, interrupted, finished)
├─ GradeType.php         (ongoing, final, behaviour)
├─ GradeEventType.php    (test, project, exam, other)
├─ NoteType.php          (positive, negative, neutral)
├─ InstrumentCondition.php (excellent, good, fair, needs_repair)
├─ ReservationPurpose.php (home, club, concert, lesson, other)
├─ ReservationStatus.php (pending, approved, rejected, cancelled)
├─ EventType.php         (concert, trip, competition, exhibition, other)
├─ EventParticipantRole.php (performer, attendee, organizer, chaperone)
├─ DayOfWeek.php         (mon, tue, wed, thu, fri, sat, sun)
```

Príklad (ostatné sú rovnaké):

php

```
// app/Enums/LessonState.php
namespace App\Enums;
enum LessonState: string
{
    case Planned = 'planned';
    case Taught = 'taught';
    case Cancelled = 'cancelled';

    public function label(): string
    {
        return match($this) {
            self::Planned => 'Plánovaná',
            self::Taught => 'Odučená',
            self::Cancelled => 'Zrušená',
        };
    }
}
```

MODEL

User.php

php

```
namespace App\Models;

use App\Enums\UserRole;
use Illuminate\Foundation\Auth\User as Authenticatable;

class User extends Authenticatable
{
    protected $casts = [
        'email_verified_at' => 'datetime',
        'password'          => 'hashed',
        'role'               => UserRole::class,
        'must_change_password' => 'boolean',
        'birth_date'         => 'date',
    ];

    public function student() { return $this->hasOne(Student::class); }
    public function teacher() { return $this->hasOne(Teacher::class); }
    public function guardian() { return $this->hasOne(Guardian::class); }

    public function isMinor(): bool
    {
        return $this->birth_date && $this->birth_date->age < 18;
    }

    public function roomReservations() { return $this->hasMany(RoomReservation::class); }
    public function instrumentReservations() { return $this->hasMany(InstrumentReservation::class); }
    public function events() { return $this->belongsToMany(Event::class); }
}
```

Student.php

php

```
class Student extends Model
{
    public function user()          { return $this->belongsTo(User::class); }
    public function specialization() { return $this->belongsTo(Specialization::class); }

    public function guardians()
    {
        return $this->belongsToMany(Guardian::class, 'guardian_student')
            ->withPivot('relationship');
    }

    public function classes()
    {
        return $this->belongsToMany(SchoolClass::class, 'class_student');
    }

    public function enrollments() { return $this->hasMany(SubjectEnrollment::class); }
    public function attendances() { return $this->hasMany(Attendance::class); }
    public function grades()      { return $this->hasMany(Grade::class); }
    public function notes()       { return $this->hasMany(Note::class); }
    public function bands()       { return $this->belongsToMany(Band::class, 'band_'); }
}
```

Teacher.php

php

```
class Teacher extends Model
{
    public function user()          { return $this->belongsTo(User::class); }
    public function specialization() { return $this->belongsTo(Specialization::class); }
    public function department()     { return $this->belongsTo(Department::class); }
    public function room()           { return $this->belongsTo(Room::class); }

    public function classes()
    {
        return $this->hasMany(SchoolClass::class);
    }

    public function subjectSchoolYears() { return $this->hasMany(SubjectSchoolYear::class); }
    public function grades()             { return $this->hasMany(Grade::class); }
    public function notes()              { return $this->hasMany(Note::class); }
    public function bands()              { return $this->hasMany(Band::class); }
}
```

Guardian.php

php

```
class Guardian extends Model
{
    public function user()          { return $this->belongsTo(User::class); }
    public function students()
    {
        return $this->belongsToMany(Student::class, 'guardian_student')
            ->withPivot('relationship');
    }
}
```

Department.php

php


```
class Department extends Model
{
    public function specializations() { return $this->hasMany(Specialization::class); }
    public function subjects() { return $this->hasMany(Subject::class); }
    public function responsibleTeacher() { return $this->belongsTo(Teacher::class, 'teacher_id'); }
}
```

Specialization.php

```
php

class Specialization extends Model
{
    public function department() { return $this->belongsTo(Department::class); }
    public function students() { return $this->hasMany(Student::class); }
    public function teachers() { return $this->hasMany(Teacher::class); }
    public function subjects() { return $this->hasMany(Subject::class); }
    public function instruments() { return $this->hasMany(Instrument::class); }
}
```

Subject.php

```
php

class Subject extends Model
{
    public function department() { return $this->belongsTo(Department::class); }
    public function specialization() { return $this->belongsTo(Specialization::class); }
    public function schoolYears() { return $this->hasMany(SubjectSchoolYear::class); }
}
```

SubjectSchoolYear.php

```
php
```

```
// PredmetVŠkolskomRoku - klúčová spojovacia entita
class SubjectSchoolYear extends Model
{
    public function subject()      { return $this->belongsTo(Subject::class); }
    public function schoolYear()  { return $this->belongsTo(SchoolYear::class); }
    public function teacher()     { return $this->belongsTo(Teacher::class); }
    public function class()       { return $this->belongsTo(SchoolClass::class, 'class'); }

    public function lessons()     { return $this->hasMany(Lesson::class); }
    public function enrollments() { return $this->hasMany(SubjectEnrollment::class); }
    public function grades()      { return $this->hasMany(Grade::class); }
    public function gradeEvents() { return $this->hasMany(GradeEvent::class); }

    // Pomocná metóda: všetci zapísaní žiaci
    public function students()
    {
        return $this->hasManyThrough(Student::class, SubjectEnrollment::class,
            'subject_school_year_id', 'id', 'id', 'student_id');
    }
}
```

SchoolYear.php

```
php

class SchoolYear extends Model
{
    protected $casts = [
        'start_date' => 'date',
        'end_date'   => 'date',
        'is_active'  => 'boolean',
    ];

    public function subjectSchoolYears() { return $this->hasMany(SubjectSchoolYear::class); }
    public function classes()            { return $this->hasMany(SchoolClass::class); }

    public static function active(): self
    {
        return static::where('is_active', true)->firstOrFail();
    }
}
```

SchoolClass.php

```
php

// Názov SchoolClass - Class je rezervované slovo v PHP!
class SchoolClass extends Model
{
    protected $table = 'classes';

    public function schoolYear() { return $this->belongsTo(SchoolYear::class); }
    public function teacher()    { return $this->belongsTo(Teacher::class); }
    public function room()       { return $this->belongsTo(Room::class); }

    public function students()
    {
        return $this->hasManyToMany(Student::class, 'class_student');
    }

    public function subjectSchoolYears() { return $this->hasMany(SubjectSchoolYear::class); }
}
```

TimeSlot.php

```
php

class TimeSlot extends Model
{
    protected $casts = [
        'day_of_week' => DayOfWeek::class,
        'start_time'  => 'datetime',
        'end_time'    => 'datetime',
    ];

    public function lessons() { return $this->hasMany(Lesson::class); }
}
```

Lesson.php

```
php
```

```
class Lesson extends Model
{
    protected $casts = [
        'date' => 'date',
        'state' => LessonState::class,
    ];

    public function subjectSchoolYear() { return $this->belongsTo(SubjectSchoolYear::class); }
    public function room() { return $this->belongsTo(Room::class); }
    public function timeSlot() { return $this->belongsTo(TimeSlot::class); }

    public function attendances() { return $this->hasMany(Attendance::class); }
    public function grades() { return $this->hasMany(Grade::class); }
    public function notes() { return $this->hasMany(Note::class); }
    public function gradeEvents() { return $this->hasMany(GradeEvent::class); }
}
```

SubjectEnrollment.php

```
php

class SubjectEnrollment extends Model
{
    protected $casts = [
        'state' => EnrollmentState::class,
        'enrolled_at' => 'date',
    ];

    public function student() { return $this->belongsTo(Student::class); }
    public function subjectSchoolYear() { return $this->belongsTo(SubjectSchoolYear::class); }
}
```

Attendance.php

```
php
```

```
class Attendance extends Model
{
    protected $casts = ['state' => AttendanceState::class];

    public function student() { return $this->belongsTo(Student::class); }
    public function lesson() { return $this->belongsTo(Lesson::class); }
}
```

GradeEvent.php

```
php

// Písomka / projekt / skúška
class GradeEvent extends Model
{
    protected $casts = [
        'type' => GradeEventType::class,
        'date' => 'date',
    ];

    public function subjectSchoolYear() { return $this->belongsTo(SubjectSchoolYear::class); }
    public function lesson()            { return $this->belongsTo(Lesson::class); }
    public function grades()             { return $this->hasMany(Grade::class); }
}
```

Grade.php

```
php
```

```
class Grade extends Model
{
    protected $casts = [
        'type' => GradeType::class,
        'graded_at' => 'datetime',
    ];

    public function student() { return $this->belongsTo(Student::class); }
    public function teacher() { return $this->belongsTo(Teacher::class); }
    public function subjectSchoolYear() { return $this->belongsTo(SubjectSchoolYear::class); }
    public function lesson() { return $this->belongsTo(Lesson::class); }
    public function gradeEvent() { return $this->belongsTo(GradeEvent::class); }
}
```

Note.php

php

```
class Note extends Model
{
    protected $casts = [
        'type' => NoteType::class,
        'visible_to_guardian' => 'boolean',
    ];

    public function student() { return $this->belongsTo(Student::class); }
    public function teacher() { return $this->belongsTo(Teacher::class); }
    public function lesson() { return $this->belongsTo(Lesson::class); }
}
```

Room.php

php

```
class Room extends Model
{
    protected $casts = ['reservable' => 'boolean'];

    public function roomReservations() { return $this->hasMany(RoomReservation); }
    public function lessons() { return $this->hasMany(Lesson::class); }
    public function instruments() { return $this->hasMany(Instrument::class); }
    public function events() { return $this->hasMany(Event::class); }

    public function scopeReservable($query)
    {
        return $query->where('reservable', true)->where('is_active', true);
    }
}
```

Instrument.php

```
php

class Instrument extends Model
{
    protected $casts = ['condition' => InstrumentCondition::class];

    public function specialization() { return $this->belongsTo(Specialization); }
    public function room() { return $this->belongsTo(Room::class); }
    public function instrumentReservations() { return $this->hasMany(InstrumentReservation); }
}
```

RoomReservation.php

```
php
```

```

class RoomReservation extends Model
{
    protected $casts = [
        'purpose' => ReservationPurpose::class,
        'status'   => ReservationStatus::class,
        'start_at' => 'datetime',
        'end_at'   => 'datetime',
    ];

    public function room() { return $this->belongsTo(Room::class); }
    public function reservedBy() { return $this->belongsTo(User::class, 'reserved_by'); }

    public function scopeConflicting($query, $start, $end, $excludeId = null)
    {
        return $query->where('start_at', '<', $end)
            ->where('end_at', '>', $start)
            ->when($excludeId, fn($q) => $q->where('id', '!=', $excludeId))
    }
}

```

InstrumentReservation.php

```

php

class InstrumentReservation extends Model
{
    protected $casts = [
        'purpose' => ReservationPurpose::class,
        'status'   => ReservationStatus::class,
        'start_at' => 'datetime',
        'end_at'   => 'datetime',
    ];

    public function instrument() { return $this->belongsTo(Instrument::class); }
    public function reservedBy() { return $this->belongsTo(User::class, 'reserved_by'); }

    public function scopeConflicting($query, $start, $end, $excludeId = null)
    {
        return $query->where('start_at', '<', $end)
            ->where('end_at', '>', $start)
            ->when($excludeId, fn($q) => $q->where('id', '!=', $excludeId))
    }
}

```

Band.php

```
php

class Band extends Model
{
    public function teacher() { return $this->belongsTo(Teacher::class); }
    public function students() { return $this->belongsToMany(Student::class, 'band_students'); }
    public function events() { return $this->belongsToMany(Event::class, 'event_bands'); }
}
```

Event.php

```
php

class Event extends Model
{
    protected $casts = [
        'type' => EventType::class,
        'starts_at' => 'datetime',
        'ends_at' => 'datetime',
        'is_public' => 'boolean',
    ];

    public function room() { return $this->belongsTo(Room::class); }
    public function bands() { return $this->belongsToMany(Band::class, 'event_bands'); }

    public function participants()
    {
        return $this->belongsToMany(User::class, 'event_participants')
            ->withPivot('role', 'note');
    }
}
```

Závislost migrácií (poradie)

```
users
school_years
departments
specializations (→ departments)
```

```
rooms
teachers      (→ users, specializations, departments, rooms)
students      (→ users, specializations)
guardians     (→ users)
guardian_student (→ guardians, students)
classes       (→ school_years, teachers, rooms)
class_student  (→ classes, students)
subjects      (→ departments, specializations)
subject_school_years (→ subjects, school_years, teachers, classes)
time_slots
lessons       (→ subject_school_years, rooms, time_slots)
subject_enrollments (→ students, subject_school_years)
attendances   (→ students, lessons)
grade_events   (→ subject_school_years, lessons)
grades        (→ students, teachers, subject_school_years, lessons,
grade_events)
notes         (→ students, teachers, lessons)
instruments    (→ specializations, rooms)
room_reservations (→ rooms, users)
instrument_reservations (→ instruments, users)
bands         (→ teachers)
band_student   (→ bands, students)
events        (→ rooms)
event_participants (→ events, users)
event_band     (→ events, bands)
```